

ShopEz

Full Stack MERN E-Commerce Application

PROJECT TECHNICAL DOCUMENTATION

Developed by: JASTI RAGHAVENDRA

Email: jsvpsairaghavendra@gmail.com

Team ID: _____

Date: 18 June 2026

1. Introduction

Project Title: ShopEz

Developer: JASTI RAGHAVENDRA (Solo Project)

ShopEz is an all-encompassing full-stack digital commerce platform engineered to deliver a fluid and frictionless purchasing journey. Utilizing the robust MERN ecosystem (MongoDB, Express.js, React.js, and Node.js), the application addresses the requirements of modern digital consumers through a highly responsive and aesthetically refined interface that prioritizes accessibility and ease of use.

2. Project Overview

Core Objectives: The fundamental goal of ShopEz is the development of a resilient and scalable marketplace architecture. It enables users to explore diverse product segments, manage virtual inventory in a personal cart, and execute financial transactions with high security. Furthermore, it incorporates a comprehensive administrative management layer for overseeing stock levels and order fulfillment.

Key Features:

- **User Management:** Secure registration, JWT-based authentication protocols, and personalized account profiles.
- **Inventory Discovery:** Categorized product navigation (Men, Women, Kids) accompanied by dynamic search and filtering logic.
- **Transaction Flow:** Real-time synchronization for shopping carts and support for Cash on Delivery (COD) processing.
- **Management Console:** A centralized dashboard for administrators to facilitate product CRUD operations and logistics monitoring.

3. System Architecture

Frontend (React.js):

The user interface is constructed using a Component-Based Architecture to ensure modularity and code reusability. Application state management is orchestrated via the React Context API alongside local hooks for granular data control. Navigation within the SPA is managed through React Router DOM to maintain deep linking and smooth transitions.

Backend (Node.js & Express.js):

The server-side infrastructure is built as an MVC-patterned RESTful API. Express.js is utilized for route orchestration, while custom middleware layers manage security auditing, data validation, and authentication checks. The backend serves aggregated JSON payloads to the client interface for dynamic rendering.

Database (MongoDB):

Data persistence is achieved via a NoSQL document-oriented model. The primary data collections include:

- **Users:** Orchestrates member credentials, contact metadata, and role-based access levels (User vs. Admin).
- **Product Catalog:** Contains comprehensive item details including descriptions, pricing matrices, and inventory counts.
- **Orders:** Maintains a historical record of all transactions, containing product references and shipping lifecycle status.

4. Setup Instructions

Prerequisites:

- Node.js environment (v18.x or above recommended)
- Dependency management via NPM or Yarn
- MongoDB Community Server accessible locally (default port: 27017)

Installation:

1. Repository Initialization:

```
git clone https://github.com/jsvpraghavendra/jastiraghavendra
```

2. Server Configuration:

```
cd server && npm install
```

3. Client Configuration:

```
cd client && npm install
```

4. Configuration Environment: Construct a `.env` configuration file within the server root:

```
PORT=5000  
MONGO_URI=mongodb://localhost:27017/shopez  
JWT_SECRET=your_secret_key
```

5. Folder Structure

Directory	Description
<code>/server/models</code>	Mongoose-based schemas and data models defining the application's core entity logic.
<code>/server/routes</code>	Defines the RESTful API endpoints and associated controller mappings.
<code>/client/src/components</code>	Modular UI elements including navigation bars, footers, and interactive product cards.
<code>/client/src/pages</code>	Primary view templates for the Home view, Product Details, and Transaction Checkout.

6. Running the Application

To initiate the development environment, execute the following commands in separate terminal sessions:

Initiate Backend:

```
cd server && npm run dev
```

Initiate Frontend:

```
cd client && npm start
```

7. API Documentation

Method	Endpoint	Description
POST	<code>/api/users/login</code>	Validates member credentials and returns a secure JWT access token.
GET	<code>/api/products</code>	Retrieves the complete product inventory list with optional filters.
POST	<code>/api/orders</code>	Initializes a new order request (Authenticated access only).

8. Authentication & Authorization

Security in ShopEz is built upon JSON Web Tokens (JWT) for stateless session handling. Upon a successful login, the server issues a signed token which the client

stores locally (e.g., LocalStorage or Cookies). Subsequent requests attach this token within the 'Authorization' header. A dedicated middleware verifies token validity and enforces role-based permissions, such as preventing standard users from accessing administrative routes.

9. User Interface

The application's UI follows a minimalist and intuitive design logic to maximize user satisfaction. It features adaptive grids for product display and a streamlined, multi-stage checkout sequence to reduce cart abandonment. Large, high-resolution media placeholders are utilized throughout to showcase merchandise effectively.

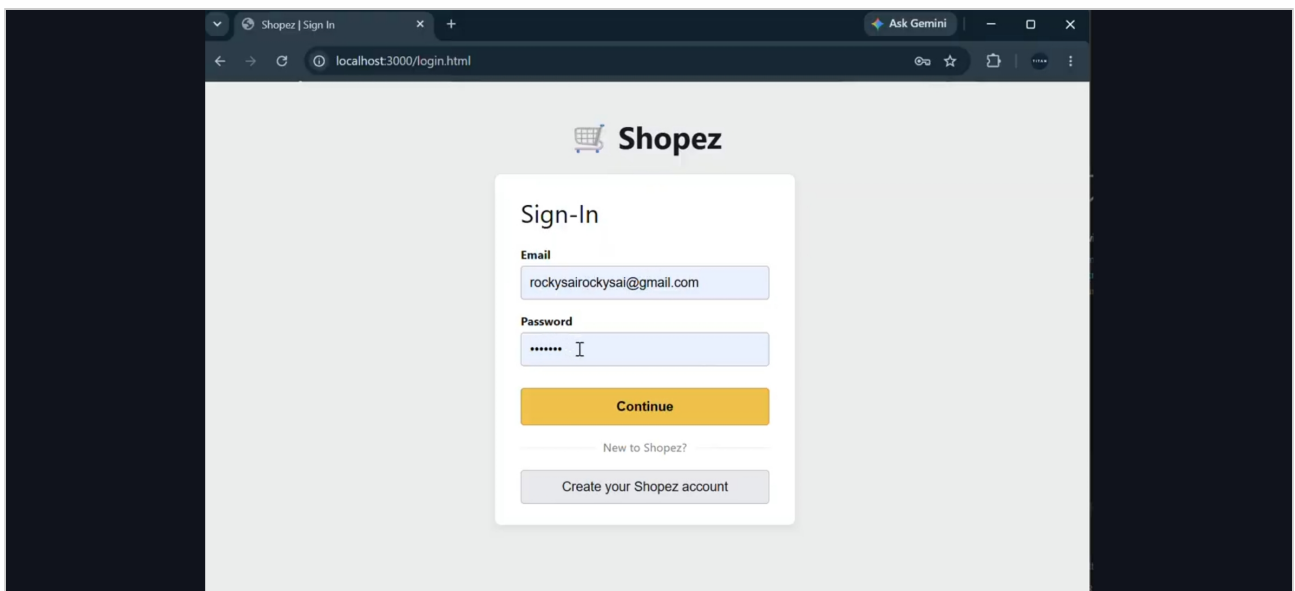
10. Testing Strategy

The development cycle involved a rigorous testing approach to ensure long-term stability:

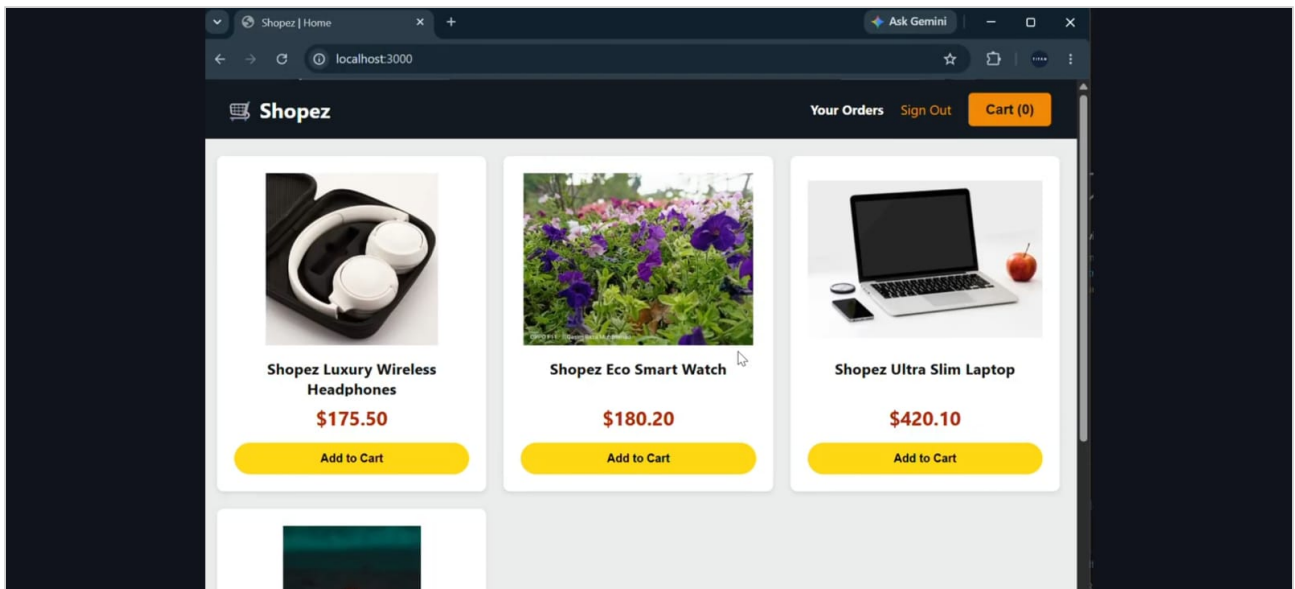
- **Unit Testing:** Isolated verification of helper logic and input validation routines.
- **Integration Testing:** Ensuring the Express routing layer communicates effectively with the MongoDB persistence layer.
- **Operational Testing:** Validating complex user flows like cart management and stock level integrity.

11. Screenshots / Demo

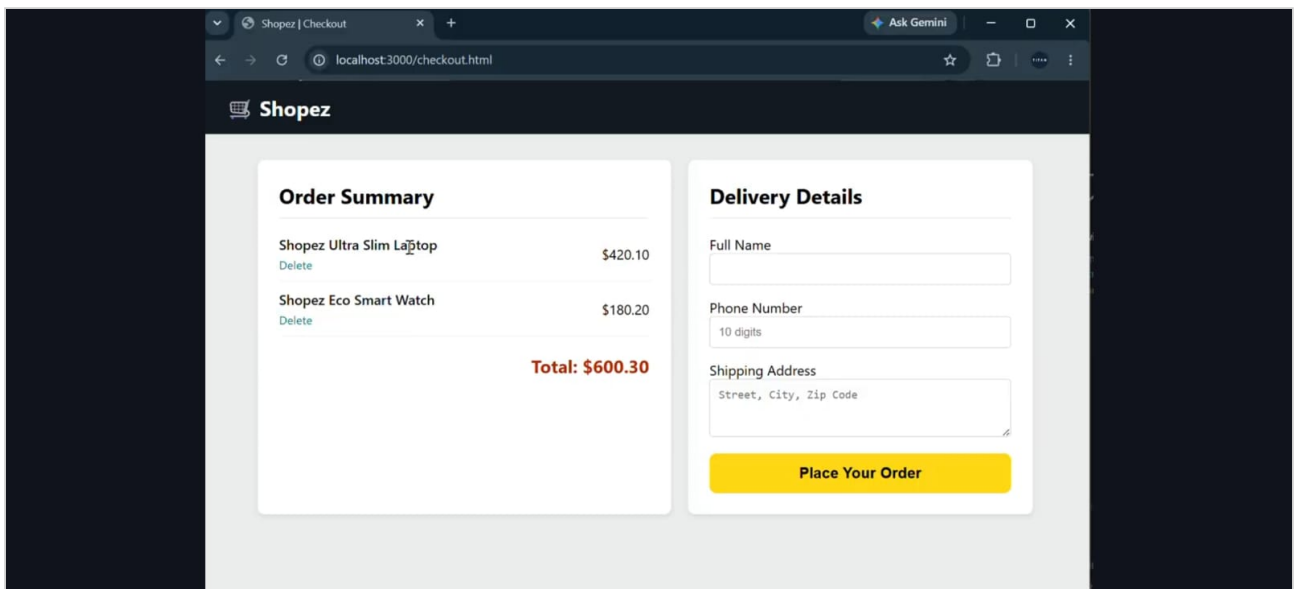
Project Repository / Live Demo Link: <https://github.com/jsvpraghavendra/jasti-raghavendra>



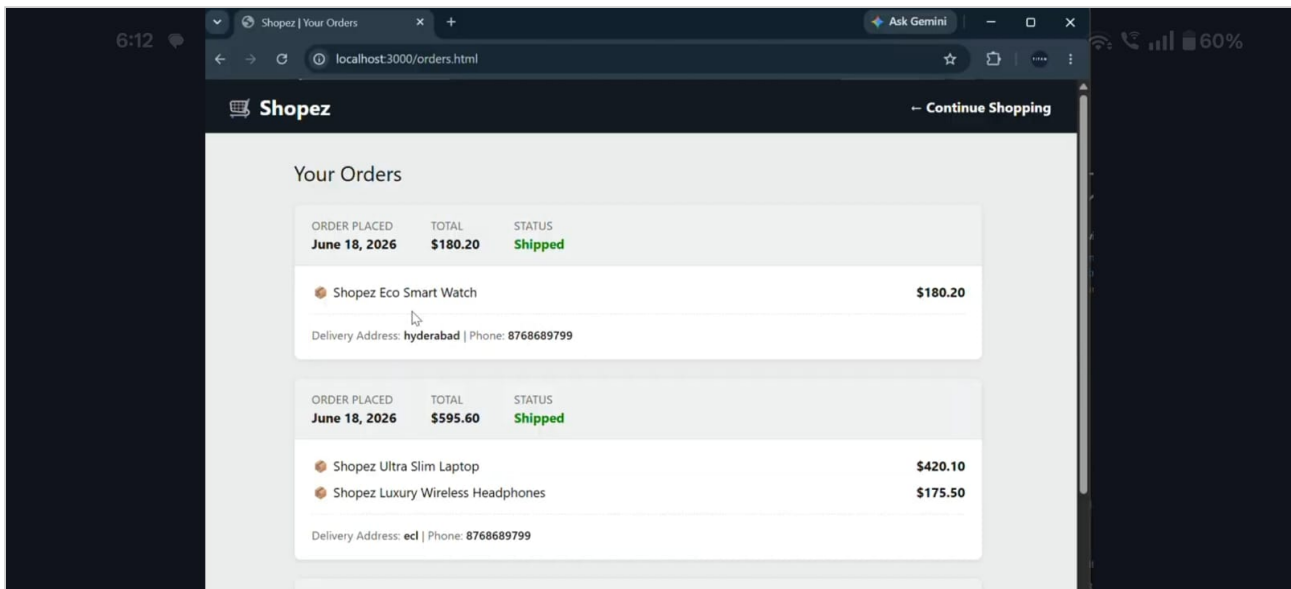
Storefront Homepage showing product listings



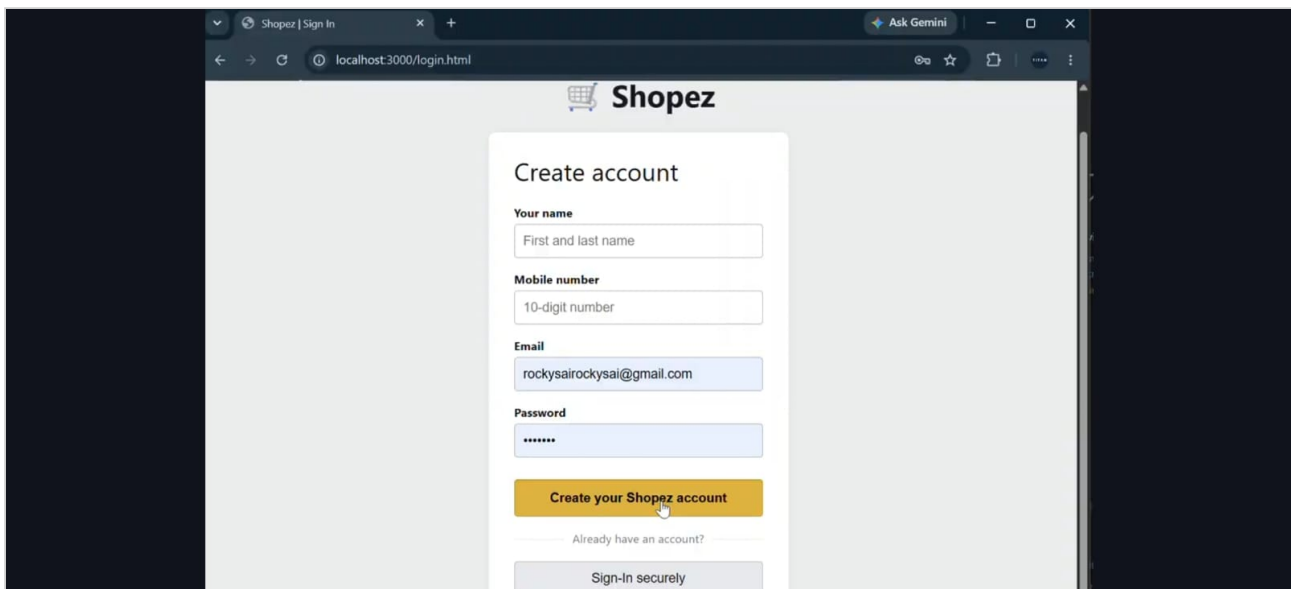
Checkout Page: Order Summary and Delivery Details



User Orders History and Status Tracking



Authentication: Sign-In Interface



User Registration: Create Account Interface

12. Known Issues

- Filters on the client-side may occasionally require a manual refresh for state synchronization.
- Large-scale file uploads currently utilize local storage; a migration to distributed cloud assets is recommended for production performance.

13. Future Enhancements

- Integration of digital payment gateways (Stripe or Razorpay) for direct card transactions.
- Implementation of an AI-driven engine for personalized item recommendations.
- Conversion to a Progressive Web App (PWA) to enable offline catalog browsing functionality.